CHAPTER 25

CubeSat control system

25.1. Space story

In 2010 Princeton Satellite Systems established a CubeSat club for middle-school students at the John Witherspoon Middle School in Princeton, New Jersey, USA. It was an after-school club open to all students. We ran the club in the school shop classroom. The club ran from 2010 through 2012. The students divided themselves into subsystem groups. The students built CubeSat hardware and wrote CubeSat software in MATLAB® and C. The shop teacher and the science teacher, Steve Carson, were the faculty members. Eloisa de Castro, a Princeton Satellite Systems engineer, gave a talk about the club at a National Science Foundation meeting.

Fig. 25.1 shows magnetic torquers built by the students. The winding rig was built by the shop teacher. Fig. 25.2 shows a solar-cell simulator and reaction-wheel hardware connected to a simulation.

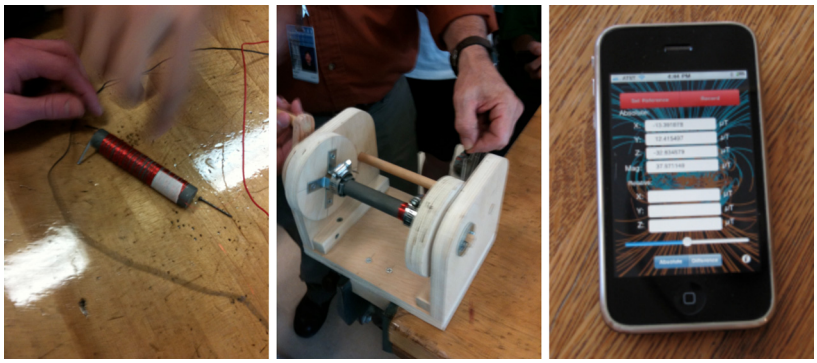


Figure 25.1 Student-built magnetic torquers and field-measurement app on an iPhone.

The club had a visit from Bob Cenker, who orbited Earth on the Space Shuttle Columbia. The students had the chance to ask Bob plenty of questions about his experience as an astronaut and they even had a discussion about orbital mechanics.

The club was a big success. Most students stayed for all four years. After the club ended, parents called us to ask if we could restart it. Today, there are many CubeSat activities for elementary, middle, and high-school students.

https://doi.org/10.1016/B978-0-32-399915-1.00037-1

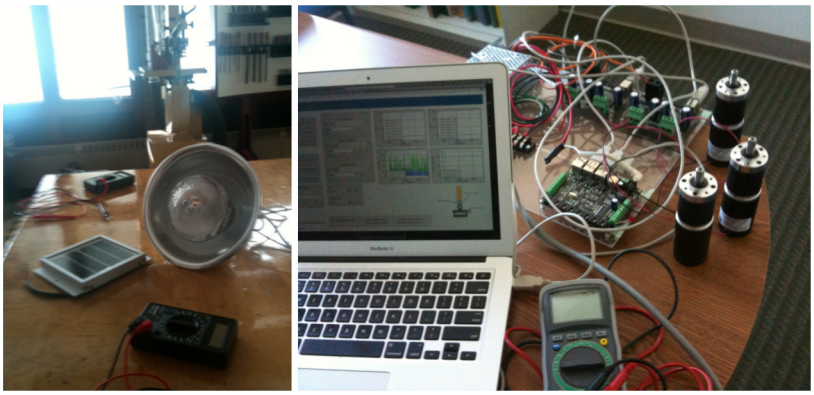


Figure 25.2 A solar cell and simulator and reaction-wheel hardware connected to a simulation.

25.2. Introduction

CubeSats are used for a wide variety of applications. The control systems range from simple gravity-gradient systems to elaborate three-axis stabilized designs with reaction wheels and star trackers. Many companies supply low-cost hardware for CubeSat applications. CubeSats are often used in LEO applications, but have been used for cutting-edge missions, like recent CubeSats that were part of a Mars mission.

This chapter describes a CubeSat control-system design. The design uses three reaction wheels for attitude control. An Earth sensor and magnetometer are used for sensing. Gyros are used as part of a Kalman filter to filter the sensor measurements. Magnetic torquers are used to unload momentum from the reaction wheels.

25.3. Requirements

The CubeSat is Earth pointing. The control requirements are to acquire the Earth and then maintain pointing.

25.4. Actuator and sensor selection

There are four attitude control devices

1. Magnetic torquers;
2. Fisheye Earth sensor;
3. Magnetometer;
4. Reaction wheels.

The concept of operations is shown in Fig. 25.3. This gives the sequence of events from separation from the launch vehicle to the operational configuration.

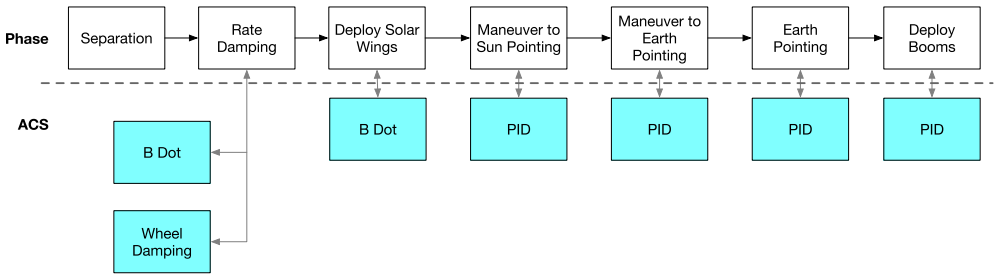


Figure 25.3 CubeSat concept of operations. Damping of rates is done at separation. The spacecraft deploys its solar wings, then reorients.

The CubeSat separates in an arbitrary orientation with arbitrary body rates. The spacecraft has two damping choices. If the body rates are low enough, it can absorb the momentum in the wheels and then proceed with operations. That would be the fastest approach. If the momentum is too high, it applies the B Dot algorithm, found in Section 12.11.1, using the magnetic torquers. The advantage of B Dot is that only the magnetometer and the magnetic torquers are needed. This has one major advantage over using the gyros because the gyro rates may have a bias, making it difficult to fully damp the body rates. Without external angle measurements, it is not possible to estimate the bias or correct for bias random walk.

25.5. Design

The CubeSat is shown in Fig. 25.4. The solar wings are 30 cm by 10 cm. They are body fixed.

25.6. Control-system design

The control system is shown in Fig. 25.5. It has a magnetometer, Earth sensor, reaction wheels, and magnetic torquers. The reaction wheels provide 3-axis control. The magnetic torquers provide momentum unloading. They can also be used for backup attitude control.

The control system uses a proportional-derivative (PD) controller. This will not completely cancel constant torque offsets. A constant disturbance torque will result in a constant attitude error.

$$\frac{T}{\theta_y} = K\left(1 + \frac{\tau_r s}{\tau_f s + 1}\right) \tag{25.1}$$

where T is the control torque, θ_y the pitch angle, τ_r the rate time constant, K is the forward gain, τ_f the rate filter time constant, and s is the derivative, i.e., $\frac{d}{dt}$. The rate filter

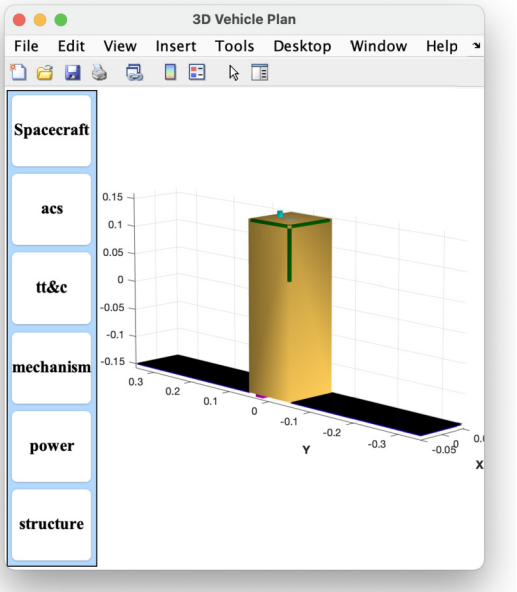


Figure 25.4 3U CubeSat with deployable solar wings. The wings are fixed. The orthogonal torquers and Earth sensor are on the nadir deck. The magnetometer is on the zenith deck.

allows the derivative to be implemented in state-space form. It also provides filtering as the differentiator amplifies noise. The wheels can also be used just for damping, in which case the control algorithm is

$$\frac{T}{\theta_y} = K\left(\frac{\tau_r s}{\tau_f s + 1}\right) \tag{25.2}$$

Attitude determination is done using an Earth sensor and magnetometer. This provides 3-axis information throughout the orbit. It requires knowing the orbital position, for the magnetic-field model, so a GPS receiver, or a reasonably accurate onboard ephemeris, is required. The momentum-control system uses a PI controller. The integral term removes any momentum offset due to steady inertial torques.

$$\frac{T}{H} = K\left(1 + \frac{1}{\tau_i s}\right) \tag{25.3}$$

where H is the inertial angular momentum. The closed-loop momentum equation is

$$\left(s + K\left(1 + \frac{1}{\tau_i s}\right)\right) H = T_d \tag{25.4}$$

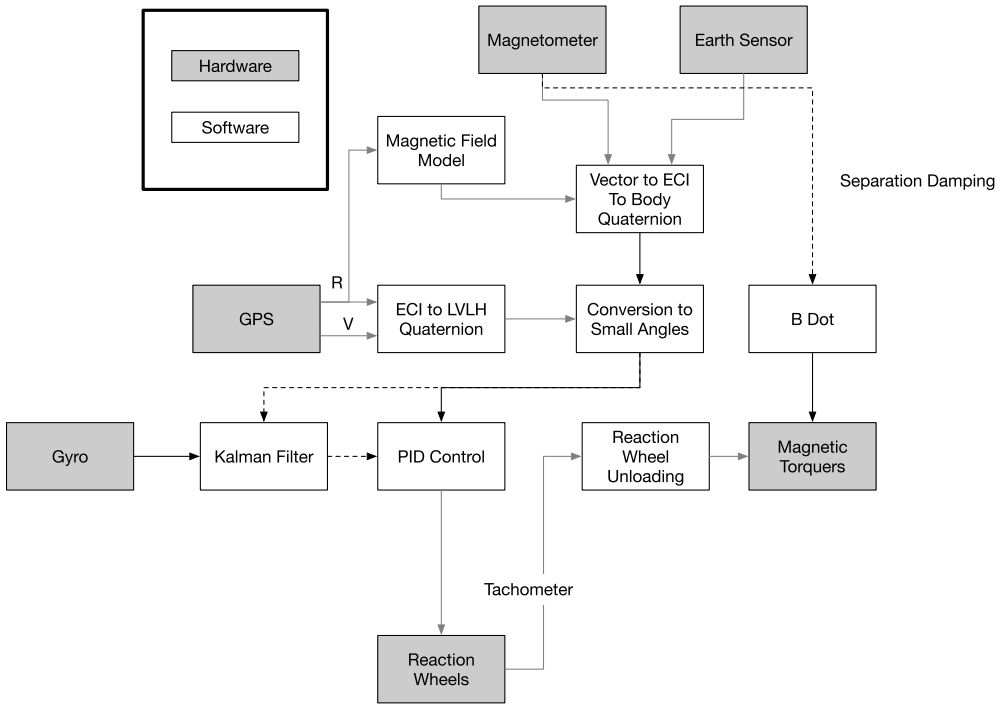


Figure 25.5 The CubeSat control system. It has a magnetometer, Earth sensor, reaction wheels, and magnetic torquers.

without the integral term, the offset is

$$H = \frac{T_d}{K} \tag{25.5}$$

With the integral term

$$\left(s^2 + K\left(s + \frac{1}{\tau_i}\right)\right) H = sT_d \tag{25.6}$$

the response is

$$H = \frac{sT_d}{s^2 + Ks + \frac{1}{\tau_i}} \tag{25.7}$$

The derivative in the numerator means that there will be no momentum offset due to a constant disturbance torque, T_d.

25.7. Attitude determination

The attitude determination system uses a single frame of data to get the quaternion from ECI to body. The CubeSat magnetometer always produces a vector measurement, but the Earth sensor only works when the Earth is in the field-of-view. The attitude determination can return either the ECI to body quaternion or ECI to LVLH quaternion. For this reason, one Earth sensor is mounted on the zenith face and one on the nadir face. This insures that a nadir vector can be measured at all times.

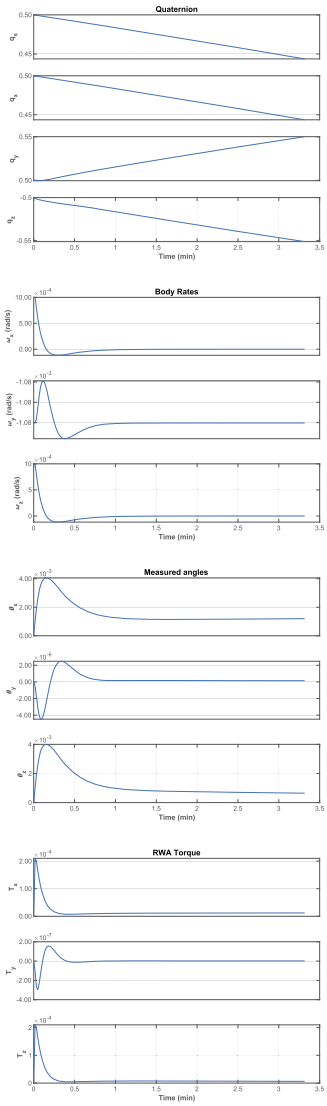
25.8. Simulation

Example 25.1 shows the attitude control system simulation. The quaternion from ECI to body changes throughout the simulation because the spacecraft, which is aligned with the LVLH frame, is always rotating with respect to the ECI frame. The body rate is nominally constant about pitch.

The control system employs direct angle measurements in the LVLH frame.

The script in Example 25.1 allows the initial angular rate to be easily changed and for a disturbance torque to be added. These allow the disturbance response of the control system to be tested. The plots in this example show the transient response from an initial rate error and the rate-damping performance of the PD controller.

```
secInDay   = 86400;
controlOn  = true;
mmOn       = false; % Momentum management
n          = 200; % Number of simulation steps
dT         = 1.0; % Time step (s)
omegaC     = 0.1; % Control bandwidth
zetaC      = 1; % Control damping ratio
jD0        = Date2JD([2023 4 4]);

% Initial states
el         = [7000 0 0 0 0 0]; % LEO
[r,v]      = El2RV(el);
orbRate    = 2*pi/Period(el(1));
omega      = [0;-orbRate;0]; % rad/s
dOmega     = [0.001;0;0.001]; % Delta angular rate
omegaRWA   = [0;0;0]; % rad/s
torqueD    = [0;0;0]*1e-6; % Disturbance torque
eSANoise   = [1e-5;1e-5]*1e-10; % ESA noise
bNoise     = 1.e-12; % Magnetometer noise
b          = [0;0;0]; % IMU bias at start
imuAngle   = [0;0;0]; % IMU angle at start
q          = QLVLH(r,v);
x          = [r;v;q;omega+dOmega;omegaRWA;b;
              imuAngle];
dRHS       = RHSReactionWheelWithOrbit;

%% Simulate
xP         = zeros(length(x)+15,n);
% Edit the control system data structure
dC         = ACSCubeSat;
dC.inr     = dRHS.inr;
dC.dT      = dT;
dC.omega   = omega; % The LVLH rate
dC.uRWA    = dRHS.uRWA;
dC.mmOn    = mmOn;
dC.kF      = dRHS.inr(2,2)*omegaC^2;
dC.tauR    = 2*omegaC*zetaC*dRHS.inr(2,2)/dC.kF;

% Create a time vector
t  = (0:n-1)*dT;
jD = jD0 + t/secInDay;

% Simulation loop
for k = 1:n
  r            = x(1:3);
  v            = x(4:6);
  qECIToBody   = x(7:10);
  % Sensor measurements
  bMeas        = QForm(qECIToBody,BDipole(r,jD(k))) +
                 bNoise*randn(3,1);
  dC.angle     = AngleDetermination(r,v,Unit(-QForm(
                 qECIToBody,r)),bMeas,jD(k));
  % Update the controller
  dC.qECIToBody = qECIToBody;
  if( controlOn )
    [dC,tThruster,tRWA] = ACSCubeSat(x,dC);
  else
    tThruster = [0;0;0]; tRWA = [0;0;0];
  end
  dRHS.torque = tThruster + torqueD; % Nm
  dRHS.torqueRWA = tRWA;
  [~,hECI] = RHSReactionWheelWithOrbit(x,0,dRHS);
  xP(:,k) = [x;bMeas*1e9;dC.angle;tThruster;tRWA;
             hECI];
  x = RK4(@RHSReactionWheelWithOrbit,x,dT,0,dRHS)
      ;
end

%% Plotting
yL = {'x_□(km)'  'x_□(km)' 'x_□(km)',...
'v_x_□(km/s)' 'v_y_□(km/s)' 'v_z_□(km/s)', ...
'q_s' 'q_x' 'q_y' 'q_z' ...
'\omega_x_□(rad/s)' '\omega_y_□(rad/s)'...
'\omega_z_□(rad/s)' ...
'\Omega_1' '\Omega_2' '\Omega_3' ...
'\theta_{ix}' '\theta_{iy}' '\theta_{iz}'...
'bias_{ix}' 'bias_{iy}' 'bias_{iz}'...
'b_x_□(nT)'  'b_y_□(nT)'  'b_z_□(nT)' ...
'\theta_x', '\theta_y', '\theta_z',...
'T_x' 'T_y' 'T_z' 'T_x' 'T_y' 'T_z'...
'H_x' 'H_y' 'H_z' };

tL = {'Position' 'Velocity' 'Quaternion'...
'Body□Rates' 'Wheel□Rates' 'IMU'...
'Magnetic□Field□Body□Frame' 'Measured□angles'...
'REA□Torque' 'RWA□Torque' 'H□ECI' };
kL = {1:3 4:6 7:10 11:13 14:16 17:22 23:25 26:28
      29:31 32:34 35:37};
for j = 1:length(tL)
  k = kL{j};
  TimeHistory(t,xP(k,:),yL(k),tL{j});
end

Figui
```



Example 25.1: Attitude control simulation.